

git # Documento de Diseño Orientado a Objetos

Proyecto: Adivina en qué estoy pensando

Árbol de Decisión Interactivo

1. Clases creadas

El proyecto usa cuatro clases principales: `Nodo`, `ArbolDecision`, `ManejadorArchivos` y `Ventana`.

2. Responsabilidad de cada clase

****Nodo****

Representa cada elemento del árbol. Puede ser un nodo de pregunta (tiene dos hijos) o un nodo de respuesta (es una hoja sin hijos).

****ArbolDecision****

Contiene la lógica del juego. Se encarga de crear el árbol inicial, recorrerlo según las respuestas del usuario, y agregar nuevas preguntas y respuestas cuando el sistema falla.

****ManejadorArchivos****

Se ocupa de todo lo relacionado con archivos. Convierte el árbol a un diccionario para guardarlo en JSON y lo reconstruye al cargarlo.

****Ventana****

Maneja toda la interfaz gráfica usando Tkinter. Muestra las pantallas, botones, formularios y mensajes. Llama a las otras clases pero no contiene lógica del árbol.

3. Atributos principales

****Nodo:****

- `texto` — el texto de la pregunta o la respuesta
- `es\_pregunta` — True si es nodo de pregunta, False si es respuesta final
- `hijo\_si` — referencia al nodo hijo de la rama Sí
- `hijo\_no` — referencia al nodo hijo de la rama No

****ArbolDecision:****

- `raiz` — el nodo raíz del árbol

****ManejadorArchivos:****

- (no tiene atributos propios, solo métodos)

****Ventana:****

- `ventana` — objeto Tk principal
- `arbol` — instancia de ArbolDecision
- `manejador` — instancia de ManejadorArchivos
- `ruta\_archivo` — ruta del archivo donde se guarda el árbol
- `nodo\_actual` — el nodo donde está la partida en este momento
- `nodo\_padre` — el nodo anterior al actual (necesario para aprender)
- `llego\_por\_si` — indica si se llegó al nodo actual por la rama Sí o No

4. Métodos principales

****Nodo:****

- ``__init__(texto, es_pregunta)`` — crea el nodo con texto y tipo

****ArbolDecision:****

- ``__init__()`` — llama a `crear_arbol_defecto` al iniciar

- ``crear_arbol_defecto()`` — crea el árbol básico (¿Es un animal? → perro/computadora)

- ``esta_en_hoja(nodo)`` — retorna True si el nodo es una hoja

- ``avanzar(nodo, respuesta)`` — retorna el hijo Sí o No según la respuesta

- ``aprender(padre, nodo_incorrecto, respuesta_correcta, nueva_pregunta, nueva_va_en_si, llego_por_si)`` — inserta nueva pregunta y respuesta en el árbol

****ManejadorArchivos:****

- ``nodo_a_dict(nodo)`` — convierte un nodo y sus hijos a diccionario (recursivo)

- ``dict_a_nodo(datos)`` — reconstruye un nodo desde diccionario (recursivo)

- ``guardar(raiz, ruta)`` — guarda el árbol completo en un archivo JSON

- ``cargar(ruta)`` — carga y reconstruye el árbol desde un archivo JSON

****Ventana:****

- ``mostrar_pantalla_inicio()`` — pantalla principal con opciones

- ``iniciar_partida()`` — empieza desde la raíz del árbol

- ``mostrar_pregunta()`` — muestra la pregunta actual o llama a `manejar_hoja`

- ``responder_si()`` / ``responder_no()`` — avanzan en el árbol

- ``manejar_hoja()`` — muestra el intento de adivinanza

- ``adivino_bien()`` — pantalla de victoria

- ``adivino_mal()`` — formulario de aprendizaje

- ``guardar_aprendizaje()`` — valida, aprende y guarda automáticamente

- ``cargar_desde_archivo()`` — abre explorador de archivos y carga árbol

- ``guardar_manual()`` — guarda manualmente con explorador de archivos

- ``nueva_partida()`` — reinicia volviendo al menú principal

- `limpiar\_pantalla()` — elimina todos los widgets actuales

5. Relación entre clases

...

Ventana

- └─ usa → ArbolDecision
- | └─ contiene → Nodo (estructura de árbol)
- └─ usa → ManejadorArchivos
 - └─ trabaja con → Nodo (para guardar y cargar)

...

`Ventana` crea una instancia de `ArbolDecision` y una de `ManejadorArchivos`. La clase `ArbolDecision` trabaja con objetos `Nodo` para formar el árbol. `ManejadorArchivos` también trabaja con `Nodo` para serializarlo y reconstruirlo.

6. Cómo se representa el árbol

El árbol está formado por objetos `Nodo` enlazados entre sí. Cada nodo tiene una referencia a su hijo izquierdo (`hijo\_si`) y a su hijo derecho (`hijo\_no`). Si un nodo es una hoja (`es\_pregunta = False`), sus hijos son `None`. El árbol completo se accede desde la `raiz` de `ArbolDecision`.

7. Cómo se recorre el árbol

Al iniciar una partida, `nodo\_actual` apunta a la raíz. Cada vez que el usuario responde Sí o No, se llama al método `avanzar()` del árbol, que retorna el hijo correspondiente. Ese hijo pasa a ser el nuevo `nodo\_actual`. Esto se repite hasta que `esta\_en\_hoja()` retorna True, momento en que el sistema intenta adivinar.

8. Cómo se agrega una nueva pregunta y respuesta

Cuando el sistema falla, `adivino\_mal()` muestra un formulario. El usuario ingresa:

1. La respuesta correcta en la que estaba pensando
2. Una pregunta que diferencia esa respuesta de la incorrecta
3. Si para la nueva respuesta la respuesta a esa pregunta es Sí o No

Con esos datos, `aprender()` en `ArbolDecision` crea un nuevo nodo de pregunta que reemplaza la hoja incorrecta. La nueva hoja correcta queda en la rama indicada, y la hoja incorrecta anterior queda en la otra rama. Si el padre era `None`, la nueva pregunta pasa a ser la raíz.

9. Cómo se guarda el árbol

`ManejadorArchivos.guardar()` llama a `nodo\_a\_dict()` que recorre el árbol de forma recursiva y convierte cada nodo en un diccionario de Python. Ese diccionario se guarda en disco usando `json.dump()`. El archivo resultante es un JSON legible que contiene toda la estructura del árbol.

El guardado ocurre automáticamente cada vez que el sistema aprende algo nuevo (dentro de `guardar\_aprendizaje()`). También existe un botón para guardar manualmente desde la pantalla de inicio.

10. Cómo se carga desde archivo

`ManejadorArchivos.cargar()` abre el archivo JSON seleccionado por el usuario, lee su contenido, y llama a `dict\_a\_nodo()` que recorre el diccionario de forma recursiva y reconstruye cada `Nodo`. Si el archivo está vacío, dañado o tiene un formato inesperado, la función retorna `None` y la ventana muestra un mensaje de error y usa el árbol por defecto.